

Relatorio Tecnico - CareHub API - Fase 3

- [CareHub API](#)
 - [Relatório Técnico de Entrega](#)
 - [Índice](#)
 - [1. Descrição da Arquitetura](#)
 - [1.1 Visão Geral](#)
 - [1.2 Arquitetura em Camadas](#)
 - [1.3 Stack Tecnológico](#)
 - [1.4 Fluxos Principais](#)
 - [2. Modelagem de Entidades e Relacionamentos](#)
 - [2.1 Modelo do Serviço de Agendamento](#)
 - [2.2 Principais Entidades](#)
 - [2.3 Modelo do Serviço de Histórico](#)
 - [3. Endpoints e Consultas Disponíveis](#)
 - [3.1 REST — Identificação do Usuário](#)
 - [3.2 REST — Listar Consultas](#)
 - [3.3 REST — Buscar Consulta por ID](#)
 - [3.4 REST — Criar Consulta](#)
 - [3.5 REST — Atualizar Consulta](#)
 - [3.6 GraphQL — Histórico por Paciente](#)
 - [3.7 GraphQL — Consultas Futuras por Paciente](#)
 - [4. Segurança, Autorização e Validação](#)
 - [4.1 Autenticação](#)
 - [4.2 Autorização por Perfil](#)
 - [4.3 Tratamento de Erros](#)
 - [4.4 Restrições de Banco](#)
 - [5. Comunicação Assíncrona](#)
 - [5.1 Publicação de Eventos](#)
 - [5.2 Consumo no Histórico](#)
 - [5.3 Consumo na Notificação](#)
 - [6. Coleção Postman](#)
 - [7. Estrutura do Banco de Dados](#)
 - [7.1 Banco do Agendamento](#)
 - [7.2 Banco do Histórico](#)
 - [8. Guia de Execução com Docker Compose](#)
 - [9. Testes Automatizados](#)
 - [9.1 Cobertura automatizada \(JaCoCo\)](#)

CareHub API

Relatório Técnico de Entrega

Versão: 1.0

Data: 10 de Setembro de 2026

Projeto: FIAP Tech Challenge - Fase 3

Stack: Java 21 | Spring Boot 4.1 | PostgreSQL | RabbitMQ | GraphQL | MapStruct | Docker

Link do repositório: <https://github.com/docarmoj/fiap-tech-challenge-fase3>

Índice

1. Descrição da Arquitetura
2. Modelagem de Entidades e Relacionamentos
3. Endpoints e Consultas Disponíveis
4. Segurança, Autorização e Validação
5. Comunicação Assíncrona
6. Coleção Postman
7. Estrutura do Banco de Dados
8. Guia de Execução com Docker Compose
9. Testes Automatizados
10. Pendências e Evoluções Recomendadas

1. Descrição da Arquitetura

1.1 Visão Geral

O CareHub é um backend modular voltado ao contexto hospitalar, com foco em agendamento de consultas, consulta ao histórico do paciente e geração de notificações assíncronas. O repositório foi organizado como **monorepo com três serviços Spring Boot independentes**, cada um com responsabilidade específica.

Serviços da solução

Serviço	Porta	Responsabilidade
carehub-agendamento	8081	

Serviço	Porta	Responsabilidade
		autenticação HTTP Basic, autorização por perfil e API REST de consultas
carehub-historico	8082	persistência do histórico consumido por eventos e API GraphQL
carehub-notificacao	8083	consumo de eventos e geração de lembretes

Infraestrutura de apoio

Componente	Uso
PostgreSQL	persistência dos serviços agendamento e historico
RabbitMQ	entrega assíncrona dos eventos de criação e atualização de consulta
Docker Compose	inicialização da infraestrutura compartilhada

1.2 Arquitetura em Camadas

Os serviços seguem arquitetura em camadas:

- **Controller / GraphQL Controller:** recebe requisições HTTP ou GraphQL
- **Service:** orquestra casos de uso e integração entre domínio, persistência e mensageria
- **Repository:** acesso a dados via Spring Data JPA
- **Model / Entity:** mapeamento das tabelas e encapsulamento das invariantes do domínio
- **Mapper:** transformação entre entidades e DTOs com MapStruct
- **Config / Security / Messaging:** configuração de segurança, filas e beans Spring

1.3 Stack Tecnológico

Componente	Versão	Descrição
Java	21	linguagem principal
Spring Boot	4.1.0	framework base
Spring Web MVC	4.1.0	endpoints REST
Spring Security	4.1.0	autenticação e autorização
Spring Data JPA	4.1.0	persistência ORM

Componente	Versão	Descrição
Spring for GraphQL	4.1.0	consultas GraphQL no histórico
Spring AMQP	4.1.0	integração com RabbitMQ
MapStruct	1.6.3	mapeamento entre entidades e DTOs
Flyway	11.x	migrations de banco
PostgreSQL	16	banco relacional no runtime local
H2	2.4.x	banco em memória nos testes
Lombok	-	redução de boilerplate
Docker Compose	-	provisionamento local

1.4 Fluxos Principais

Fluxo REST de agendamento

Cliente HTTP → ConsultaController → ConsultaService → Consulta / ConsultaRepository → PostgreSQL

Fluxo assíncrono

carehub-agendamento → ConsultaEventFactory → ConsultaEventPublisher → RabbitMQ → carehub-notificacao / carehub-historico

Fluxo GraphQL

Cliente GraphQL → HistoricoGraphQLController → ConsultaHistoricoMapper → ConsultaHistoricoService → ConsultaHistoricoRepository → PostgreSQL do histórico

2. Modelagem de Entidades e Relacionamentos

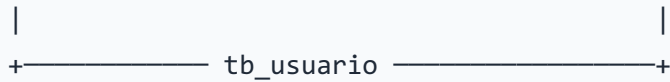
2.1 Modelo do Serviço de Agendamento

O serviço de agendamento mantém o domínio principal da aplicação.

```

tb_paciente (1) ———< tb_consulta >——— (1) tb_profissional
      ^                                   ^

```



2.2 Principais Entidades

Paciente (tb_paciente)

Representa o paciente atendido pela instituição.

Atributo	Tipo	Descrição
id	Long	identificador
nome	String	nome do paciente
cpf	String	CPF único
email	String	e-mail único
telefone	String	telefone de contato

Profissional (tb_profissional)

Representa médicos e enfermeiros.

Atributo	Tipo	Descrição
id	Long	identificador
nome	String	nome do profissional
registroProfissional	String	CRM/COREN
tipo	Enum	MEDICO ou ENFERMEIRO
especialidade	String	especialidade ou área

Usuario (tb_usuario)

Credencial de autenticação associada a um paciente ou profissional.

Atributo	Tipo	Descrição
id	Long	identificador

Atributo	Tipo	Descrição
username	String	login
password	String	senha criptografada com BCrypt
role	Enum	MEDICO, ENFERMEIRO, PACIENTE
ativo	boolean	habilita ou bloqueia acesso
paciente_id	Long	vínculo para paciente
profissional_id	Long	vínculo para profissional

Consulta (tb_consulta)

Representa a consulta agendada.

Atributo	Tipo	Descrição
id	Long	identificador
paciente_id	Long	paciente da consulta
profissional_id	Long	profissional responsável
data_hora	LocalDateTime	data e hora da consulta
status	Enum	AGENDADA, REALIZADA, CANCELADA
observacoes	String	informações complementares

Além do mapeamento estrutural, a entidade Consulta passou a encapsular o comportamento principal do agregado:

- criação de um novo agendamento com status inicial AGENDADA
- atualização controlada dos dados da consulta

2.3 Modelo do Serviço de Histórico

O serviço carehub-historico mantém uma projeção própria do histórico consumido por eventos, desacoplada do banco do agendamento.

tb_usuario

tb_consulta_historico

- consulta_id
- paciente_id
- paciente_nome
- profissional_id
- profissional_nome
- data_hora
- status
- observacoes
- atualizado_em

Esse desenho favorece leitura rápida no GraphQL e evita dependência direta do banco transacional do agendamento.

3. Endpoints e Consultas Disponíveis

3.1 REST — Identificação do Usuário

GET /usuarios/me → 200 OK

Retorna o usuário autenticado e seus vínculos de domínio.

Exemplo de resposta

```
{
  "username": "medico1",
  "role": "MEDICO",
  "profissionalId": 1
}
```

3.2 REST — Listar Consultas

GET /consultas → 200 OK

Comportamento por perfil:

- MEDICO e ENFERMEIRO: listam consultas de todos os pacientes
- PACIENTE: lista apenas as próprias consultas

Exemplo de resposta

```
[
  {
```

```
"id": 1,
"pacienteId": 1,
"pacienteNome": "Joao Silva",
"profissionalId": 1,
"profissionalNome": "Dr. Carlos Eduardo",
"dataHora": "2025-03-10T09:00:00",
"status": "REALIZADA",
"observacoes": "Consulta de rotina - retorno em 6 meses"
}
]
```

3.3 REST — Buscar Consulta por ID

GET /consultas/{id} → 200 OK | 403 Forbidden | 404 Not Found

- profissionais podem consultar qualquer consulta
- pacientes só podem consultar consultas próprias

3.4 REST — Criar Consulta

POST /consultas → 201 Created | 400 Bad Request

Disponível para MEDICO e ENFERMEIRO.

Request Body

```
{
  "pacienteId": 1,
  "profissionalId": 1,
  "dataHora": "2030-12-01T09:00:00",
  "observacoes": "Consulta criada pelo Postman"
}
```

Validações

- pacienteId obrigatório
- profissionalId obrigatório
- dataHora obrigatória e futura
- observacoes com no máximo 1000 caracteres

3.5 REST — Atualizar Consulta

PUT /consultas/{id} → 200 OK | 400 Bad Request | 404 Not Found

Disponível para MEDICO e ENFERMEIRO.

Request Body

```
{
  "pacienteId": 1,
  "profissionalId": 1,
  "dataHora": "2030-12-02T14:00:00",
  "status": "REALIZADA",
  "observacoes": "Atualizada pelo Postman"
}
```

Validações

- pacienteId obrigatório
- profissionalId obrigatório
- dataHora obrigatória e futura
- status obrigatório
- observacoes com no máximo 1000 caracteres

3.6 GraphQL — Histórico por Paciente

POST /graphql

Query disponível no serviço carehub-historico.

```
query {
  historicoPorPaciente(pacienteId: "1") {
    id
    dataHora
    status
    observacoes
    paciente {
      id
      nome
    }
    profissional {
      id
      nome
    }
  }
}
```

3.7 GraphQL — Consultas Futuras por Paciente

```
query {  
  consultasFuturasPorPaciente(pacienteId: "1") {  
    id  
    dataHora  
    status  
  }  
}
```

Regras de acesso

- MEDICO e ENFERMEIRO consultam qualquer paciente
- PACIENTE consulta apenas o próprio histórico

4. Segurança, Autorização e Validação

4.1 Autenticação

Os serviços carehub-agendamento e carehub-historico utilizam **HTTP Basic** com Spring Security.

Credenciais de carga inicial:

- medico1 / 123456
- enfermeiro1 / 123456
- paciente1 / 123456
- paciente2 / 123456
- inativo1 / 123456

4.2 Autorização por Perfil

Perfil	Permissões
MEDICO	visualizar e editar consultas; consultar histórico
ENFERMEIRO	registrar e editar consultas; consultar histórico
PACIENTE	visualizar apenas as próprias consultas e o próprio histórico

4.3 Tratamento de Erros

Nos endpoints HTTP, erros de autenticação, autorização e validação retornam `application/problem+json`.

Exemplo de erro de validação

```
{
  "type": "about:blank",
  "title": "Erro de validação",
  "status": 400,
  "detail": "A requisição contém campos inválidos.",
  "instance": "/consultas",
  "errors": {
    "pacienteId": "pacienteId é obrigatório",
    "dataHora": "dataHora deve estar no futuro"
  }
}
```

4.4 Restrições de Banco

As migrations também reforçam integridade:

- username, cpf, email e registro_profissional únicos
- check constraint para coerência entre role e vínculo de domínio
- índices em chaves estrangeiras e data_hora

5. Comunicação Assíncrona

O fluxo assíncrono foi organizado com separação explícita entre produtor e consumidores:

- carehub-agendamento usa apenas `carehub.rabbitmq.producer.exchange` e `carehub.rabbitmq.producer.routing-key`
- carehub-historico e carehub-notificacao usam propriedades próprias em `carehub.rabbitmq.consumer.*`
- filas, DLQ e bindings ficaram restritos aos consumidores, evitando acoplamento indevido no serviço produtor

O evento publicado continua representando criação e atualização de consultas, mas sua montagem passou a ser responsabilidade do componente dedicado `ConsultaEventFactory`.

5.1 Publicação de Eventos

Sempre que uma consulta é criada ou alterada, o serviço carehub-agendamento publica um evento no RabbitMQ.

Campos relevantes do evento

```
{
  "consultaId": 10,
  "pacienteId": 1,
  "nomePaciente": "Joao Silva",
  "emailPaciente": "joao.silva@email.com",
  "profissionalId": 1,
  "nomeProfissional": "Dr. Carlos Eduardo",
  "dataHora": "2030-12-01T09:00:00",
  "status": "AGENDADA",
  "observacoes": "Consulta criada",
  "acao": "CONSULTA_CRIADA",
  "ocorridoEm": "2026-09-05T13:47:00"
}
```

5.2 Consumo no Histórico

O carehub-historico consome a fila carehub.historico.queue e persiste o estado mais recente da consulta em tb_consulta_historico.

Características:

- atualização idempotente por consulta_id
- descarte de eventos antigos via campo atualizado_em
- leitura otimizada para GraphQL

5.3 Consumo na Notificação

O carehub-notificacao consome a fila carehub.notificacoes.queue e gera lembretes com assunto e mensagem variando conforme a ação (CONSULTA_CRIADA ou CONSULTA_ALTERADA).

Atualmente o envio é feito por **log/console**.

6. Coleção Postman

A collection está em:

docs/carehub.postman_collection.json

Cobertura incluída:

- autenticação
- matriz de autorização REST
- criação e atualização de consultas
- testes do GraphQL do histórico

Variáveis principais

Variável	Valor padrão
baseUrl	http://localhost:8081
baseUrlHistorico	http://localhost:8082
senha	123456

7. Estrutura do Banco de Dados

7.1 Banco do Agendamento

Tabelas principais:

- tb_paciente
- tb_profissional
- tb_usuario
- tb_consulta

Carga inicial:

- 3 pacientes
- 2 profissionais
- 5 usuários de autenticação
- 4 consultas base

7.2 Banco do Histórico

Tabelas principais:

- tb_usuario
- tb_consulta_historico

O histórico possui carga inicial própria para manter consistência dos testes e permitir consulta GraphQL mesmo antes de novos eventos no broker.

8. Guia de Execução com Docker Compose

O arquivo `docker-compose.yml` sobe:

- RabbitMQ
- PostgreSQL

Subir infraestrutura

```
docker compose up -d
```

Subir serviços

```
cd carehub-agendamento  
mvnw.cmd spring-boot:run
```

```
cd carehub-historico  
mvnw.cmd spring-boot:run
```

```
cd carehub-notificacao  
mvnw.cmd spring-boot:run
```

9. Testes Automatizados

Os módulos possuem testes unitários e de integração cobrindo segurança, mensageria e consultas. Os testes de integração foram organizados com perfil próprio e dependem de RabbitMQ disponível no ambiente local.

O projeto possui testes cobrindo:

- autenticação HTTP Basic
- autorização por perfil e vínculo
- criação e atualização de consultas
- publicação e consumo de eventos
- queries GraphQL do histórico

9.1 Cobertura automatizada (JaCoCo)

A cobertura foi validada com JaCoCo após a execução dos testes em cada módulo do monorepo.

Para repetir a execução localmente sem divergências de ambiente:

- use Java 21 no terminal e no Maven Wrapper
- mantenha o RabbitMQ disponível para os testes de integração
- evite executar a suíte com Java 25 enquanto o agente JaCoCo do ambiente não estiver alinhado com esse runtime

Serviço	Instruções	Linhas	Branches	Status
carehub-agendamento	94,81%	97,61%	88,89%	testes passando
carehub-historico	82,16%	82,83%	61,54%	testes passando
carehub-notificacao	80,38%	80,39%	100,00%	testes passando
Consolidado	87,96%	89,56%	73,91%	monorepo